

# Service + Validações + Tratamento de Erros

Refatoração + regras de negócio + respostas de erro padronizadas.

Camada Service

Bean Validation

Erros padronizados

CRUD completo

# O que vamos fazer hoje

1

## Por que refatorar? Controller vs Service

O problema de ter lógica de negócio no Controller

2

## Criando a camada Service

@Service, injeção por construtor, @Transactional

3

## Bean Validation — validando a entrada

@NotBlank, @Size, @Email, @Valid no @RequestBody

4

## Tratamento global de erros

ResourceNotFoundException + GlobalExceptionHandler

5

## CRUD completo — POST, PUT, DELETE

Completar os verbos HTTP usando Service + validação + erros

6

## Teste end-to-end no Swagger

Verificar 404, 400 e os 5 verbos funcionando

# Por que tirar a lógica do Controller?

## ✗ ANTES — Controller faz tudo

ProdutoController.java — sem Service

```
@PostMapping
public ResponseEntity<Produto> criar(
    @RequestBody Produto produto) {

    // ✗ lógica de negócio no controller
    if (produto.getNome() == null ||
        produto.getNome().isBlank()) {
        return ResponseEntity
            .badRequest().build();
    }
    if (repo.existsByNome(produto.getNome()))
        return ResponseEntity.status(409).build();

    return ResponseEntity.status(201)
        .body(repo.save(produto));
}
```

## ✓ DEPOIS — Controller delega

ProdutoController.java — com Service

```
@PostMapping
public ResponseEntity<Produto> criar(
    @Valid @RequestBody Produto produto) {

    // ✓ controller só delega e responde
    Produto salvo = service.salvar(produto);

    return ResponseEntity
        .status(201).body(salvo);
}

// Toda a lógica foi para:
// ProdutoService.salvar()
```



Regra: Controller recebe HTTP - chama Service - devolve resposta. Nunca toma decisões de negócio.

# Criando a camada Service

service/ProdutoService.java

```
@Service
@RequiredArgsConstructor // Lombok: DI por construtor
public class ProdutoService {
    private final ProdutoRepository repository;
    @Transactional(readOnly = true)
    public Iterable<Produto> listarTodos() {
        return repository.findAll(); }
    @Transactional(readOnly = true)
    public Produto buscarPorId(Long id) {
        return repository.findById(id).orElseThrow(() ->
            new ResourceNotFoundException("Produto", id)); }
    @Transactional
    public Produto salvar(Produto produto) {
        if (repository.existsByNomeIgnoreCase(produto.getNome()))
            throw new ValidationException("Nome duplicado");
        return repository.save(produto); }
```

## Anotações-chave

### @Service

Marca como bean Spring gerenciado

### @RequiredArgsConstructor

Lombok gera construtor com final fields (DI)

### @Transactional(readOnly)

Otimiza leituras: sem lock, sem flush

### @Transactional

Atomicidade: rollback automático em exceção

### orElseThrow()

Lança exceção se não encontrado — nunca null

### existsByNomeIgnoreCase()

Query method: verifica duplicata antes de salvar

# Atualizando o Controller para usar o Service

O Controller deixa de acessar o Repository diretamente e passa a delegar tudo ao Service:

controller/ProdutoController.java — versão Semana 3

```
@RestController
@RequestMapping("/api/v1/produtos")
@RequiredArgsConstructor
public class ProdutoController {
    private final ProdutoService service; // ← Service, NÃO Repository

    @GetMapping
    public ResponseEntity<Iterable<Produto>> listar() {
        return ResponseEntity.ok(service.listarTodos());
    }

    @GetMapping("/{id}")
    public ResponseEntity<Produto> buscar(@PathVariable Long id) {
        return ResponseEntity.ok(service.buscarPorId(id));
    }

    // ... POST, PUT, DELETE vêm no Slide 9
}
```



## Troque o campo

private final ProdutoRepository → private final  
ProdutoService



## Remova a lógica

Todo if/else de negócio sai do controller — vai para o  
Service



## Mantenha simples

Controller só: receber, delegar, montar ResponseEntity  
e retornar

# Validando dados de entrada com anotações

entity/Produto.java – declare as regras aqui

```
@Entity
@Data @NoArgsConstructor @AllArgsConstructor
public class Produto {

    @Id @GeneratedValue(strategy=IDENTITY)
    private Long id;

    @NotBlank(message="Nome obrigatório")
    @Size(min=2, max=150)
    @Column(nullable=false, length=150)
    private String nome;
```

controller – dispere com @Valid

```
@PostMapping
public ResponseEntity<Produto> criar(
    @Valid @RequestBody Produto produto) {
    // ↑ @Valid aciona as anotações da entity
    //   falha → lança MethodArgumentNotValidException
    return ResponseEntity.status(201)
        .body(service.salvar(produto));
}
```

## Anotações disponíveis

**@NotNull**

Campo não pode ser null

**@DecimalMin**

Valor decimal com escala

**@NotBlank**

String não vazia (sem só espaços)

**@Email**

Formato de e-mail válido

**@NotEmpty**

Coleção/string com ao menos 1 item

**@Pattern**

Validação via regex

**@Size**

Tamanho min/max (string ou lista)

**@Past/@Future**

Data antes / depois de hoje

**@Min / @Max**

Valor numérico mínimo / máximo

**@Digits**

Inteiro e casas decimais máximos

# ResourceNotFoundException e ErrorResponse

Crie 2 arquivos no pacote exception/ — são a base do tratamento de erros:

```
exception/ResourceNotFoundException.java
```

```
// Exceção para recurso não encontrado (404)
public class ResourceNotFoundException
    extends RuntimeException {

    public ResourceNotFoundException(
        String resource, Long id) {
        super(resource + " não encontrado: " + id);
    }
}
```

```
exception/ErrorResponse.java
```

```
// Corpo padronizado de todas as respostas de erro
public record ErrorResponse(
    int status,
    String message,
    LocalDateTime timestamp
) {
    public ErrorResponse(int status, String msg) {
        this(status, msg, LocalDateTime.now());
    }
}
```

## Antes vs Depois

### ✗ ANTES

```
HTTP 500
"trace": "java.lang.NullPointerException
        at com.example... [50 linhas]"
// impossível de depurar pela equipe
```

### ✓ DEPOIS

```
HTTP 404
{
    "status": 404,
    "message": "Produto não encontrado: 99",
    "timestamp": "2025-05-18T10:30:00"
}
// claro, padronizado, consumível pelo front
```

# GlobalExceptionHandler — centralizando todos os erros

Um único arquivo captura TODAS as exceções da API e devolve respostas padronizadas:

```
exception/GlobalExceptionHandler.java

@ControllerAdvice // intercepta exceções de TODOS os controllers
public class GlobalExceptionHandler {

    // — 404: recurso não encontrado —
    @ExceptionHandler(ResourceNotFoundException.class)
    public ResponseEntity<ErrorResponse> notFound(
        ResourceNotFoundException ex) {
        return ResponseEntity.status(404)
            .body(new ErrorResponse(404, ex.getMessage()));
    }

    // — 400: validação falhou (@Valid) —
    @ExceptionHandler(MethodArgumentNotValidException.class)
    public ResponseEntity<ErrorResponse> validation(
        MethodArgumentNotValidException ex) {
        String msg = ex.getBindingResult()
            .getFieldErrors().stream()
            .map(f -> f.getField()+" : "+f.getDefaultMessage())
            .collect(Collectors.joining(" | "));
        return ResponseEntity.badRequest()
            .body(new ErrorResponse(400, msg));
    }
}
```

## Como o Spring usa isso

- 1 Controller recebe a requisição
- 2 @Valid dispara Bean Validation
- 3 Exceção é lançada (ex: 404)
- 4 Spring intercepta com @ControllerAdvice
- 5 Handler correto é invocado
- 6 ErrorResponse JSON é devolvido



Adicione quantos  
@ExceptionHandler quiser para  
cobrir diferentes tipos de erro.

# Adicionando POST, PUT e DELETE ao Controller

controller/ProdutoController.java – CRUD completo

```
// — POST: criar —————
@PostMapping
public ResponseEntity<Produto> criar(@Valid @RequestBody Produto produto) {
    return ResponseEntity.status(201).body(service.salvar(produto));
}

// — PUT: atualizar —————
@PutMapping("/{id}")
public ResponseEntity<Produto> atualizar(
    @PathVariable Long id, @Valid @RequestBody Produto dados) {
    return ResponseEntity.ok(service.atualizar(id, dados));
}

// — DELETE: remover —————
@DeleteMapping("/{id}")
public ResponseEntity<Void> deletar(@PathVariable Long id) {
    service.deletar(id);
    return ResponseEntity.noContent().build(); // 204
}
```

**POST**

→ 201 Created

**PUT**

→ 200 OK

**DELETE**

→ 204 No Content

# Métodos atualizar() e deletar() no Service

service/ProdutoService.java — completando

```
// — Atualizar —  
@Transactional  
public Produto atualizar(Long id, Produto dados) {  
    // 1. Busca (lança 404 se não existir)  
    Produto atual = buscarPorId(id);  
  
    // 2. Atualiza só os campos permitidos  
    atual.setNome(dados.getNome());  
    atual.setPreco(dados.getPreco());  
    atual.setDescricao(dados.getDescricao());  
  
    // 3. Salva e retorna atualizado  
    return repository.save(atual);  
}  
  
// — Deletar —  
@Transactional  
public void deletar(Long id) {  
    // Garante 404 antes de tentar deletar  
    Produto produto = buscarPorId(id);  
    repository.delete(produto);  
}
```

## Boas práticas

### buscarPorId() reutilizado

Não duplique a lógica de busca. Um método, um lugar.

### Só campos permitidos

Nunca faça atual = dados. Escolha quais campos atualizar.

### Delete via objeto

repository.delete(obj) em vez de deleteById(id) — confirma a existência antes.

### @Transactional nos dois

Atomicidade: se qualquer passo falhar, o banco não é alterado.

### Sem deletar em cascata cego

Cuidado com CascadeType.REMOVE — entenda o impacto no domínio.

# Testando os 5 verbos no Swagger

Abra localhost:8080/swagger-ui.html e execute cada operação em ordem:

**POST****/api/v1/produtos**

```
{ "nome": "Mouse",  
  "preco": 89.90 }
```

**→ 201 Created + body com ID gerado****POST****/api/v1/produtos**

```
{ "nome": "", "preco": 0 }
```

**→ 400 Bad Request + mensagem de validação****GET****/api/v1/produtos**

(sem body)

**→ 200 OK + lista com o produto criado****GET****/api/v1/produtos/999**

(sem body)

**→ 404 Not Found + JSON padronizado****PUT****/api/v1/produtos/1**

```
{ "nome": "Mouse Novo",  
  "preco": 99.9 }
```

**→ 200 OK + body atualizado****DELETE****/api/v1/produtos/1**

(sem body)

**→ 204 No Content — sem body na resposta**

# Checklist da Semana 3

Tudo marcado antes de encerrar a

☐ Camada Service criada para cada entidade principal

☐ Controllers chamam Service, não Repository diretamente

☐ @Transactional(readOnly) nos métodos de leitura

☐ @Transactional nos métodos de escrita (salvar, atualizar, deletar)

☐ Bean Validation declarado nas entidades (@NotBlank, @Size...)

☐ @Valid usado em todos os @RequestBody do Controller

☐ ResourceNotFoundException criada no pacote exception/

☐ ErrorResponse (record) criado no pacote exception/

☐ GlobalExceptionHandler com @ControllerAdvice implementado

☐ POST, PUT, DELETE funcionando para todas as entidades

# O que vamos construir na próxima aula

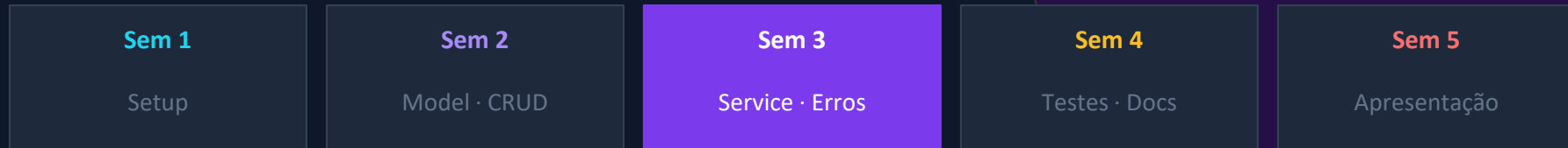
Com o CRUD funcionando e os erros tratados

|                                   |                                                                                          |
|-----------------------------------|------------------------------------------------------------------------------------------|
| <b>DTOs — Request e Response</b>  | Separe a entidade da API. ProdutoRequestDTO + ProdutoResponseDTO como Java Records.      |
| <b>Testes com @DataJpaTest</b>    | Testa Repository contra H2 em memória — sem subir o contexto web completo.               |
| <b>Testes com Mockito</b>         | ProdutoServiceTest: mocka o Repository, testa apenas a lógica de negócio. Sem banco.     |
| <b>Testes com @WebMvcTest</b>     | ProdutoControllerTest: sobe só a camada web, mocka o Service, testa rotas e status HTTP. |
| <b>Cobertura ≥ 70% com JaCoCo</b> | ./mvnw test gera relatório em target/site/jacoco/index.html. Mostre a porcentagem.       |
| <b>Swagger documentado</b>        | @Tag, @Operation e @ApiResponses em todos os controllers. Swagger vira manual da API.    |

# Semana 3 concluída.

Controller limpo. Service com lógica. Erros padronizados. CRUD completo.

## Roadmap do projeto



## Commits esperados desta semana:

```
git log --oneline (semana 3)
```

```
refactor: extract business logic to ProdutoService
feat: add @Transactional to Service read/write methods
feat: add Bean Validation to entity fields
feat: add ResourceNotFoundException and ErrorResponse
feat: add GlobalExceptionHandler with 404 and 400 handlers
feat: add POST, PUT, DELETE endpoints to all controllers
```